



Fork Infosystems

Karad — Computer Education Institute

COURSE NOTES

AI Workshop

AI Basics & AI Tools

Concepts · Prompt Engineering · Tools · Ethics · Careers

AI Basics

Prompt Engineering

AI Tools

Ethics

Careers

Ranjit Sir

Fork Infosystems, Karad · 9172135355 / 7757962804



Table of Contents

Part 1 — What is Artificial Intelligence?

1.1 Definition & Core Idea · 1.2 History & Timeline · 1.3 Types of AI · 1.4 AI vs ML vs DL vs GenAI

Part 2 — How AI Actually Works

2.1 Data — The Fuel of AI · 2.2 Training a Model · 2.3 Neural Networks · 2.4 Parameters & Weights · 2.5 Inference vs Training

Part 3 — Machine Learning Fundamentals

3.1 Supervised Learning · 3.2 Unsupervised Learning · 3.3 Reinforcement Learning · 3.4 Common Algorithms · 3.5 Overfitting & Underfitting

Part 4 — Large Language Models (LLMs)

4.1 What is an LLM? · 4.2 Transformer Architecture · 4.3 Tokens & Context Window · 4.4 Temperature & Top-P · 4.5 Popular LLMs Compared

Part 5 — Prompt Engineering

5.1 What is Prompt Engineering? · 5.2 Anatomy of a Good Prompt · 5.3 Core Prompting Techniques · 5.4 Chain-of-Thought · 5.5 Few-Shot Prompting · 5.6 System Prompts & Roles · 5.7 Prompt Dos and Don'ts · 5.8 Practical Prompt Examples

Part 6 — Important AI Concepts & Terminology

6.1 Key Terms A–Z · 6.2 Hallucination & Grounding · 6.3 RAG — Retrieval-Augmented Generation · 6.4 Fine-Tuning vs Prompting · 6.5 AI Agents & Tool Use · 6.6 Embeddings & Vector Databases · 6.7 Multimodal AI

Part 7 — AI Tools Landscape

7.1 Text & Chat AI · 7.2 Image Generation · 7.3 Audio & Speech · 7.4 Video Generation · 7.5 Code AI Tools · 7.6 Productivity & Office AI · 7.7 AI Search Engines

Part 8 — Using AI Tools — Practical Guide

8.1 ChatGPT / Claude / Gemini · 8.2 Image Tools — Midjourney & DALL-E · 8.3 Code Tools — GitHub Copilot · 8.4 Google NotebookLM · 8.5 Building an AI Workflow

Part 9 — Ethics, Safety & Responsible AI

9.1 Bias in AI · 9.2 Deepfakes & Misinformation · 9.3 Privacy Concerns · 9.4 Job Impact · 9.5 AI Safety Principles

Part 10 — AI Careers & Next Steps

10.1 Career Paths in AI · 10.2 Skills to Learn · 10.3 Learning Resources · 10.4 Suggested Projects



Part 1 — What is Artificial Intelligence?

1.1 Definition & Core Idea

Artificial Intelligence (AI) is the science of making machines perform tasks that normally require human intelligence — such as understanding language, recognising images, making decisions, translating text, writing code, and solving problems. AI does not 'think' like a human; it finds statistical patterns in large amounts of data and uses those patterns to produce useful outputs.

Simple definition: AI is a computer program that learns from examples and uses what it learned to make predictions or decisions on new data it has never seen before.

Concept	Human Analogy	AI Equivalent
Learning	A student studies textbooks	Model trains on millions of data examples
Memory	Student remembers facts	Model stores patterns in billions of parameters
Answering	Student answers exam questions	Model performs inference on new inputs
Improving	Student revises after mistakes	Model adjusts weights during training

1.2 Brief History & Timeline

Year	Milestone
1950	Alan Turing proposes the 'Turing Test' — can a machine think?
1956	Term 'Artificial Intelligence' coined at the Dartmouth Conference
1980s	Expert systems boom — AI encoded as hand-written rules
1997	IBM Deep Blue defeats chess world champion Garry Kasparov
2012	AlexNet wins ImageNet — deep learning revolution begins
2016	DeepMind AlphaGo defeats world Go champion — 10 ¹⁷ possible positions
2017	Google introduces the Transformer architecture — foundation of all modern LLMs
2020	GPT-3 released — 175 billion parameters; fluent human-level text generation
2022	ChatGPT launches — 1 million users in 5 days; fastest growing product in history
2023	GPT-4, Claude, Gemini, Llama 2 — multimodal AI goes mainstream
2024	AI Agents, reasoning models (o1), real-time video AI, on-device LLMs
2025	AI coding agents write entire codebases; agentic AI enters enterprise workflows

1.3 Types of AI

Type	Description	Current Status
Narrow AI (ANI)	Designed for ONE specific task only. Cannot generalise beyond it	EXISTS NOW — all current AI is narrow AI
General AI (AGI)	Can perform ANY intellectual task a human can. Learns and reasons like a human	DON'T EXIST YET — active research goal



Type	Description	Current Status
Super AI (ASI)	Intelligence far exceeding the best human minds in every field.	HYPOTHETICAL — topic of AI safety research
Generative AI	AI that creates new content: text, images, audio, video, code.	EXISTS NOW — ChatGPT, Midjourney, Suno, etc.
Reactive AI	Only responds to current input; no memory of past interactions.	EXISTS — chess engines, spam filters
Limited Memory AI	Uses past data (short window) to inform current decisions.	EXISTS — self-driving cars, LLMs with context window

1.4 AI vs ML vs DL vs GenAI

These terms are often confused. Think of them as nested circles — each one is a subset of the previous:

```
+-----+ | ARTIFICIAL INTELLIGENCE (broadest
term) | | Any technique making machines behave intelligently | | |
+-----+ | | MACHINE LEARNING | | | AI that
learns patterns from data | | | | | +-----+ | | |
DEEP LEARNING | | | | ML using neural networks (layers) | | | | |
+-----+ | | | | GENERATIVE AI | | | | | Deep learning
that CREATES | | | | new content (text/image...) | | | | |
+-----+ | | | | +-----+ | | |
+-----+ |
+-----+
```

Term	What it means	Example
AI	Broad field — machines doing intelligent tasks	Spam filter, Siri, Chess engine
ML	AI that learns from data without being explicitly programmed	Netflix recommendations, fraud detection
Deep Learning	ML using multi-layer neural networks	Face recognition, speech recognition
Generative AI	Deep learning models that generate new content	ChatGPT, DALL-E, Suno, Runway



Part 2 — How AI Actually Works

2.1 Data — The Fuel of AI

AI models learn entirely from data. The quality, quantity, and diversity of training data determines how good the model becomes. Garbage in = garbage out.

Data Type	Examples	Used For
Text	Books, websites, Wikipedia, code, articles	Language models (ChatGPT, Claude)
Images	Photos, drawings, medical scans, satellite	Image recognition, DALL-E, Stable Diffusion
Audio	Speech recordings, music, podcasts	Speech-to-text, music generation (Suno)
Video	YouTube clips, movies, surveillance footage	Video understanding, Sora, Runway
Tabular	CSV files, databases, spreadsheets	Predictions, recommendations, fraud detection
Code	GitHub repositories, Stack Overflow	GitHub Copilot, code generation models

GPT-4 was trained on approximately 1 trillion tokens of text — roughly equivalent to reading every book, website, and article ever written multiple times.

2.2 Training a Model

Training is the process of adjusting a model's internal numbers (weights/parameters) so it gets better at its task. The model makes predictions, compares them to correct answers, calculates the error, and adjusts weights to reduce that error. This repeats millions of times.

Training Loop: 1. Feed input data into model | v 2. Model makes a prediction | v 3. Compare prediction to correct answer --> calculate Loss (error) | v 4. Backpropagation --> adjust weights to reduce loss | v 5. Repeat millions of times until loss is minimised | v 6. Trained model ready for deployment --> Inference

2.3 Neural Networks

A neural network is a system of interconnected nodes (neurons) arranged in layers, loosely inspired by the human brain. Each connection has a weight — a number that determines how much influence one neuron has on the next.

Input Layer Hidden Layers Output Layer [x1] ----+ [x2] ----|----> [neuron] --+ [prediction]
[x3] ----|----> [neuron] --+----> [probability] [x4] ----+----> [neuron] --+ [class label] Each
neuron: $\text{sum}(\text{inputs} \times \text{weights}) \rightarrow \text{activation function} \rightarrow \text{output}$ Common activations: ReLU, Sigmoid, Softmax, GELU, SiLU

Layer Type	Purpose	Example Use
Input Layer	Receives raw data (pixels, tokens, numbers)	Image pixels, word tokens
Hidden Layers	Extract features and patterns from data	Edges, shapes, grammar, meaning
Output Layer	Produces the final prediction or generation	Class label, next word, bounding box



Layer Type	Purpose	Example Use
Attention Layer	Weighs which parts of input matter most	Core of Transformer / LLM models

2.4 Parameters & Weights

Parameters (also called weights) are the numbers inside a neural network that get adjusted during training.
More parameters = more capacity to learn complex patterns.

Model	Parameters	Context
GPT-2 (2019)	1.5 Billion	Impressive for its time — couldn't follow instructions well
GPT-3 (2020)	175 Billion	First 'emergent' abilities — could follow instructions
GPT-4 (2023)	~1 Trillion (est.)	Multimodal, professional-level reasoning
Llama 3.1 (2024)	405 Billion	Open source — runs on high-end servers
Gemma 3 (2025)	27 Billion	Runs on a single GPU or powerful laptop
Phi-3 Mini (2024)	3.8 Billion	Runs on a smartphone — efficient small model

More parameters is NOT always better. A small, well-trained model can outperform a large, poorly trained one.
Efficiency (parameters per performance) matters as much as size.

2.5 Inference vs Training

Aspect	Training	Inference
What happens	Model learns from data — weights are updated	Model uses learned weights to answer questions
When it runs	Once (or periodically) before release	Every time you send a message
Hardware	Thousands of GPUs for weeks or months	One or a few GPUs per user request
Cost	Tens of millions of dollars for large models	Fractions of a cent per query
Example	OpenAI trains GPT-4 for months	You type a prompt — GPT-4 replies in 2 seconds

Part 3 — Machine Learning Fundamentals

3.1 Supervised Learning

The model is trained on labelled data — every training example has an input AND a correct answer. The model learns to map inputs to outputs.

Sub-type	Task	Example
Classification	Predict a category / class label	Is this email spam or not spam?
Regression	Predict a continuous number	What will be tomorrow's temperature?
Object Detection	Find and label objects in an image	Detect cars and pedestrians in a photo
Sequence-to-Sequence	Map one sequence to another	Translate English to Hindi

Most real-world AI products use supervised learning. Labelling training data is often the most time-consuming and expensive part of building an AI system.

3.2 Unsupervised Learning

The model is trained on data WITHOUT labels. It discovers hidden structure, patterns, or groupings on its own.

Technique	What it Does	Example
Clustering	Groups similar data points together	K-Means: group customers by behaviour
Dimensionality Reduction	Compresses data to fewer dimensions	PCA: visualise high-dimensional data
Anomaly Detection	Finds data points that don't fit the pattern	Detect fraudulent bank transactions
Generative Models	Learns the distribution of data to create new samples	GANs, VAEs, Diffusion models

3.3 Reinforcement Learning

The model (called an Agent) learns by taking actions in an environment and receiving rewards or penalties. No labelled data — it learns by trial and error, maximising cumulative reward over time.

Agent --> takes Action --> Environment --> gives Reward/Penalty --> Agent learns
Example: Teaching AI to play chess
Action = move a chess piece
Reward = +1 for winning, -1 for losing, 0 for draw
Learning = play millions of games, remember which moves led to wins
Real-world applications: AlphaGo / AlphaZero -- mastered chess, Go, shogi from scratch
RLHF -- used to align ChatGPT to be helpful and safe
Robotics -- training robot arms to grasp objects
Recommendation -- YouTube/Netflix learning what keeps you watching

3.4 Common ML Algorithms

Algorithm	Type	Best For
Linear Regression	Supervised	Predicting continuous values (price, temperature)
Logistic Regression	Supervised	Binary classification (yes/no, spam/not spam)



Algorithm	Type	Best For
Decision Tree	Supervised	Interpretable classification and regression
Random Forest	Supervised	High-accuracy tabular data — ensemble of trees
K-Nearest Neighbours	Supervised	Simple classification by proximity
Support Vector Machine	Supervised	High-dimensional data; image classification
K-Means Clustering	Unsupervised	Customer segmentation; data grouping
Neural Network / Deep Learning	Both	Images, text, speech, complex patterns
Gradient Boosting (XGBoost)	Supervised	Best for tabular/structured data — Kaggle winner

3.5 Overfitting & Underfitting

Problem	What Happens	Analogy	Fix
Overfitting	Model memorises training data; performs well on training data but poorly on new data	Student memorises answers but can't solve new problems	More data, dropout, regularisation, simpler model
Underfitting	Model is too simple to learn the pattern; performs poorly on both training and test data	Student doesn't study enough to understand the topic	More training, more complex model, better features
Good Fit	Model learns general patterns; performs well on both training and test data	Student truly understands concepts and can apply them	Properly tune model complexity, enough training

Part 4 — Large Language Models (LLMs)

4.1 What is an LLM?

A Large Language Model (LLM) is a deep learning model trained on massive amounts of text data. It learns to predict the next word (token) in a sequence. From this one simple task, it develops the ability to write, reason, translate, summarise, code, and answer questions. Modern LLMs are trained using: (1) Pre-training on internet-scale text, (2) Fine-tuning on curated instruction datasets, (3) RLHF — Reinforcement Learning from Human Feedback to align with human values.

Key insight: LLMs don't 'know' facts the way humans do. They are extremely sophisticated pattern-completion machines. When ChatGPT answers your question, it is generating the statistically most likely continuation of your prompt based on its training data.

4.2 Transformer Architecture

Virtually all modern LLMs are built on the Transformer architecture, introduced by Google in the landmark 2017 paper 'Attention Is All You Need'. The key innovation is the **Self-Attention Mechanism** — which allows every word in a sentence to 'pay attention' to every other word simultaneously, capturing long-range dependencies far better than previous recurrent (RNN/LSTM) models.

```
Transformer Block (repeated N times in each LLM): Input Tokens | [ Token Embedding ] -- converts words to vectors | [ Positional Encoding ] -- adds word-position information | [ Multi-Head Self-Attention ] -- each token attends to all others | [ Add & Norm ] -- residual connection + normalisation | [ Feed-Forward Network ] -- processes each position independently | [ Add & Norm ] | [ Repeat x N layers ] | [ Output: probability over vocabulary ] --> next token prediction
```

4.3 Tokens & Context Window

LLMs don't process words — they process **tokens**. A token is roughly 3–4 characters or 0.75 words on average. The **context window** is the maximum number of tokens the model can 'see' at once — its working memory.

Concept	Detail	Example
Token	Basic unit of text processed by the model	'Hello' = 1 token, 'unbelievable' = 3 tokens
Tokenisation	Text split into tokens using a vocabulary (BPE)	'ChatGPT' → ['Chat','G','PT']
Context Window	Max tokens model can read in one go	GPT-4: 128K tokens ≈ 300 pages of text
Prompt Tokens	Tokens in your input message	'What is AI?' = 4 tokens
Completion Tokens	Tokens in the model's response	Response of 200 words ≈ 267 tokens
Cost	API pricing is per 1,000 tokens (input + output)	GPT-4o: ~\$2.50 per 1M input tokens

4.4 Temperature & Top-P — Controlling Creativity

These parameters control how random or deterministic the model's output is:



Parameter	Low Value	High Value	Best Use
Temperature (0.0 – 2.0)	0.0–0.3: Focused, consistent, deterministic. Low: more predictable.	0.6–2.0: Creative, varied output. Different each time.	Different each time. High: creative writing, brainstorming.
Top-P (0.0 – 1.0)	0.1–0.3: Only considers most likely tokens.	0.9–1.0: Considers full range of possible tokens.	Combine with temperature. Usually keep at 0.9 for balance.

Rule of thumb: For factual tasks (coding, data extraction, Q&A;) use Temperature 0.1–0.3. For creative tasks (stories, ideas, marketing copy) use Temperature 0.7–1.0. Never use Temperature > 1.2 for production use.

4.5 Popular LLMs Compared

Model	Company	Strengths	Access
GPT-4o	OpenAI	Best all-rounder; fast; multimodal; huge ecosystem	ChatGPT Plus / API
Claude 3.5 Sonnet	Anthropic	Long context; best for writing & analysis; safe	Claude.ai / API
Gemini 1.5 Pro	Google	1M token context; integrated with Google Workspace	Gemini.google.com
Llama 3.1 405B	Meta	Best open-source; can run locally; no API cost	HuggingFace / Ollama
Mistral Large	Mistral AI	European; strong coding; API and open weights	Mistral API
Phi-3 Mini	Microsoft	Tiny (3.8B); runs on phone; surprisingly capable	Azure / Ollama
DeepSeek R1	DeepSeek	Strong reasoning; open source; very efficient	DeepSeek API

Part 5 — Prompt Engineering

5.1 What is Prompt Engineering?

Prompt Engineering is the skill of crafting input text (prompts) to get the best possible output from an AI model. Since LLMs are pattern-completion engines, *how* you phrase your request dramatically affects the quality, accuracy, and format of the response. A well-engineered prompt can be the difference between a generic, useless answer and a precise, expert-level response — without changing the model at all.

Analogy: Prompt engineering is like knowing how to ask the right question. A vague question gets a vague answer. A precise, context-rich question gets a precise, useful answer. The model hasn't changed — only how you communicate with it.

5.2 Anatomy of a Good Prompt

A complete, effective prompt typically contains these components:

Component	Purpose	Example
Role / Persona	Tell the AI what expert it should be	"You are an experienced Python developer..."
Context / Background	Give relevant information the AI needs	"I am building a school management system..."
Task	The specific action you want performed	"Write a function that calculates student grades..."
Format	Specify how the output should be structured	"Return a numbered list / table / JSON / code block"
Constraints	Limits or requirements for the response	"Keep it under 200 words. Avoid jargon. Use simple English."
Examples	Show the model what good output looks like	"For example, input: 85 → output: 'Grade B'"
Tone	The style or voice of the response	"Explain this to a 10-year-old" or "Be formal and professional"

You don't need all components in every prompt. For simple tasks, just the task is fine. For complex or important tasks, including more components dramatically improves results.

5.3 Core Prompting Techniques

Technique	Description	When to Use
Zero-Shot	Ask directly with no examples. Model uses its training.	Simple, well-known tasks. Quick answers.
Few-Shot	Provide 2–5 examples of input→output before your actual request.	Custom formats, specific styles, non-standard tasks.
Chain-of-Thought (CoT)	Ask model to 'think step by step' before answering.	Maths, logic puzzles, multi-step reasoning.
Role Prompting	Assign the model a specific expert persona.	Technical writing, domain-specific advice.
Instruction Prompting	Give direct, explicit instructions with clear constraints.	Structured outputs, data extraction, formatting.
Iterative Refinement	Start broad, then ask follow-up prompts to improve.	Complex writing, code development, research.
Self-Consistency	Ask the same question multiple times; pick the best answer.	Important decisions, factual verification.
Tree of Thought	Ask model to explore multiple reasoning paths and choose best.	Complex problem solving, creative planning.

5.4 Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting dramatically improves performance on reasoning, mathematics, and logic tasks by asking the model to show its work step by step. This mirrors how humans solve complex problems.

Without Chain-of-Thought (poor result):

Prompt: "If a train travels 120 km in 2 hours then stops for 30 minutes then travels 180 km in 3 hours, what is the average speed?" Response: "50 km/h" <-- often incorrect for multi-step problems

With Chain-of-Thought (correct result):

Prompt: "If a train travels 120 km in 2 hours then stops for 30 minutes then travels 180 km in 3 hours, what is the average speed? Think step by step." Response: Step 1: Total distance = 120 + 180 = 300 km Step 2: Total time = 2 hours + 0.5 hours (stop) + 3 hours = 5.5 hours Step 3: Average speed = 300 / 5.5 = 54.5 km/h Answer: 54.5 km/h

Trigger phrases: 'Think step by step' / 'Let's think through this' / 'Show your reasoning' / 'Work through this carefully'

5.5 Few-Shot Prompting

Provide examples of the exact input→output pattern you want before giving your actual input. The model recognises the pattern and applies it consistently.

Prompt example (sentiment classification): Classify the sentiment as Positive, Negative, or Neutral. Review: "The food was amazing and service was excellent!" Sentiment: Positive Review: "Long wait time and cold food. Very disappointing." Sentiment: Negative Review: "The restaurant was okay. Nothing special." Sentiment: Neutral Review: "Best biryani I have ever had! Highly recommend!" Sentiment:

3–5 examples is usually the sweet spot. Too few and the pattern isn't clear; too many and you waste context window tokens.

5.6 System Prompts & Role Assignment

A **system prompt** is a hidden instruction given to the AI before the conversation starts. It sets the AI's persona, expertise, tone, and rules for the entire conversation. In APIs and custom applications, the system prompt is set by the developer.

SYSTEM PROMPT (set once, applies to whole conversation):
----- You are a helpful tutor at Fork Infossystems, Karad. Your students are aged 10-16 learning programming for the first time. Always explain concepts using simple analogies and real-life examples. Never use jargon without explaining it first. When giving code examples, always add comments explaining each line. Be encouraging and patient. If a student seems confused, rephrase.
----- USER: "What is a variable?" AI response will now be automatically tailored for young beginners.

5.7 Prompt Dos and Don'ts



DO ✓	DON'T ✗
Be specific and detailed about what you want	Write vague one-line prompts like 'write something about AI'
Specify the output format (table, list, JSON, paragraph)	Leave the format up to the model when structure matters
Provide relevant context and background	Assume the model knows your specific situation
Use role prompting for domain-specific tasks	Ask generic questions when expert-level advice is needed
Ask for step-by-step reasoning on complex tasks	Expect the model to get multi-step maths right without CoT
Iterate and refine — use follow-up prompts	Accept the first response if it's not quite right
Specify length: 'in 3 sentences' / 'in 200 words'	Complain about length without specifying what you want
Tell the model what NOT to do when it matters	Only describe what you want and forget edge cases
Verify important facts — AI can hallucinate	Blindly trust AI output for critical decisions

5.8 Practical Prompt Examples

For Students & Learning:

```
"Explain [topic] as if I am 12 years old using a real-life analogy." "I am learning Python. I don't understand recursion. Give me 3 progressively harder examples with explanations." "Quiz me on [topic] with 5 multiple-choice questions. Show answers after."
```

For Writing & Content:

```
"Write a professional email to [recipient] asking for [purpose]. Tone: polite but direct. Length: under 150 words." "Rewrite this paragraph to be clearer and more concise: [paste text]" "Generate 10 creative taglines for a computer institute in a small city."
```

For Coding:

```
"You are a senior Python developer. Review this code and identify bugs, performance issues, and suggest improvements. Code: [paste code]" "Write a Python function that [description]. Include: - Type hints - Docstring with examples - Error handling - Unit tests"
```

For Research & Analysis:

```
"Summarise the following text into 5 bullet points, each under 20 words." "Compare [option A] and [option B] in a table with columns: Feature, Option A, Option B, Winner" "List pros and cons of [topic]. Format as two columns."
```



Part 6 — Important AI Concepts & Terminology

6.1 Essential AI Terms — A to Z Glossary

Term	Definition
Agent	An AI system that can take actions, use tools, and complete multi-step tasks autonomously.
Attention Mechanism	Core component of Transformers; allows model to weigh importance of each token relative to all others.
Backpropagation	Algorithm that calculates how much each weight contributed to the error; used to update weights during training.
Benchmark	A standardised test used to measure and compare AI model performance (e.g. MMLU, HumanEval, HellaSwag).
CLIP	Contrastive Language–Image Pre-training — model that connects text and image understanding (OpenAI).
Context Window	The maximum number of tokens an LLM can process in a single request (its 'working memory').
Diffusion Model	Generative model that creates images by learning to reverse a noise-adding process (Stable Diffusion, DALL-E 3).
Embedding	A numerical vector (list of numbers) representing text, images, or other data in a way that captures meaning.
Fine-Tuning	Additional training on a specific dataset to specialise a pre-trained model for a particular task or style.
Foundation Model	A large model trained on broad data that can be adapted for many downstream tasks (GPT-4, CLIP, Whisper).
GAN (Generative Adversarial Network)	Two competing neural networks: Generator creates fake data; Discriminator tries to spot fakes.
Gradient Descent	Optimisation algorithm that minimises loss by adjusting weights in the direction of steepest decrease.
Grounding	Connecting AI outputs to verified external facts, databases, or documents to reduce hallucination.
Hallucination	When an AI confidently generates plausible-sounding but factually incorrect information.
Hyperparameters	Settings chosen before training begins (learning rate, batch size, layers) — not learned from data.
Inference	Using a trained model to generate outputs for new inputs — what happens when you use ChatGPT.
Latent Space	A compressed, multi-dimensional mathematical space where a model stores its learned representations.
LLM	Large Language Model — a deep learning model trained on large text corpora to understand and generate language.
Logit	The raw, unnormalised score a model assigns to each possible output before applying softmax.
LoRA	Low-Rank Adaptation — an efficient method to fine-tune large models by training only a small number of additional weights.
Loss Function	A mathematical measure of how wrong the model's predictions are; the model tries to minimise this during training.
Multimodal	AI that can process and generate multiple types of data — text, images, audio, and video.
Perplexity	A measure of how surprised a language model is by text. Lower = model found text more predictable.
RLHF	Reinforcement Learning from Human Feedback — humans rate model outputs; these ratings train a reward model used to
Softmax	Mathematical function that converts raw scores (logits) into probabilities that sum to 1.
Temperature	Controls randomness of model output. 0 = deterministic; high values = creative/unpredictable.
Token	The basic unit of text processed by an LLM — roughly 3–4 characters or 0.75 of a word.
Tokenisation	Process of splitting text into tokens using a vocabulary (typically Byte-Pair Encoding / BPE).
Transfer Learning	Using a model pre-trained on one task as a starting point for a different task.
Transformer	The neural network architecture (2017) underpinning virtually all modern LLMs. Uses self-attention.



Term	Definition
Vector Database	Database optimised to store and search embeddings — enables semantic search and RAG applications.
Zero-Shot	Model performs a task it has never seen examples of during inference — relies purely on training knowledge.

6.2 Hallucination & Grounding

Hallucination is when an AI model confidently produces information that is factually wrong, fabricated, or completely made up. This is one of the most important limitations to understand when working with LLMs.

Type	Example	Risk Level
Factual hallucination	States a false date, name, or statistic as fact	High — hard to detect without verification
Source fabrication	Invents citations, paper titles, or URLs that don't exist	High — looks real and credible
Reasoning error	Makes a logical mistake with total confidence	Medium — especially in maths/logic
Outdated information	Presents old data as current fact	Medium — depends on knowledge cutoff

How to reduce hallucinations:

- Always verify important factual claims from the AI against reliable sources.
- Use **Retrieval-Augmented Generation (RAG)** to ground AI in real documents.
- Ask the model to cite sources — though it may still fabricate them, it's a useful signal.
- Use lower temperature for factual tasks.
- Ask the model: 'Are you confident about this? What might you be wrong about?'

6.3 RAG — Retrieval-Augmented Generation

RAG is a technique that gives an LLM access to a knowledge base or document store at query time. Instead of relying only on its training data, the model retrieves relevant documents and uses them as context — dramatically improving accuracy and reducing hallucination.

RAG Pipeline: 1. Document Ingestion Your documents (PDFs, websites, databases) | 2. Chunking & Embedding Split into chunks --> embed each chunk to a vector | 3. Store in Vector Database (Pinecone, Chroma, Weaviate) | 4. User asks a question | 5. Embed the question --> find most similar chunks | 6. Inject retrieved chunks into the LLM prompt as context | 7. LLM answers using BOTH its training AND the retrieved docs | Result: Accurate, grounded, up-to-date answer with source references

Examples of RAG in products: ChatGPT with web browsing, Google NotebookLM, Perplexity AI. All use RAG to answer with up-to-date or document-specific information.

6.4 Fine-Tuning vs Prompting

Aspect	Prompting	Fine-Tuning
What it is	Craft clever inputs to guide model behaviour	Re-train model on new examples to change its weights
Cost	Free / per API call	Expensive — GPU time + data preparation

Aspect	Prompting	Fine-Tuning
Speed	Instant	Hours to days of training
Flexibility	Very flexible — change prompt any time	Fixed until you fine-tune again
Expertise needed	Anyone can learn prompting	Requires ML engineering skills
Best for	Custom outputs, styles, workflows	Consistent domain expertise, specific tone, efficiency
When to choose	Start here — works for 90% of use cases	When prompting is too slow or inconsistent

6.5 AI Agents & Tool Use

An **AI Agent** is an LLM that can take actions in the world: browse the web, run code, read/write files, call APIs, send emails, and use other tools — all to complete a goal with minimal human intervention. Agents can plan, reflect on their actions, and course-correct.

```
Agent Loop: Goal: 'Find the cheapest flight from Pune to Delhi next Friday' | Plan: 1. Search flight comparison sites 2. Compare prices 3. Report | Act: [web_search('flights Pune Delhi next Friday')] | Observe: [search results returned] | Reflect: 'I found 5 results. Now I should compare prices.' | Act: [analyse_data(results)] | Final Answer: 'IndiGo 6E-204 at 06:30, Rs. 3,450 – cheapest option'
```

Agent Framework	Description	Use Case
OpenAI Assistants	Built-in tool use: code interpreter, file search, web	ChatGPT plugins and GPTs
LangChain	Python framework for building custom AI agents	Developer-built AI workflows
AutoGen (Microsoft)	Multi-agent conversations — agents talk to each other	Complex automated tasks
CrewAI	Specialised role-based agents work as a team	Research, writing, analysis pipelines
Claude Computer Use	Claude can control a computer GUI directly	Automating complex desktop tasks

6.6 Embeddings & Vector Databases

An **embedding** converts text, images, or other data into a list of numbers (a vector) that captures semantic meaning. Similar concepts have vectors that are numerically close to each other in the embedding space. This enables powerful semantic search.

```
Text --> Embedding Model --> Vector (list of numbers) 'The student learned Python' --> [0.23, -0.51, 0.88, 0.12, ...] 'The pupil studied coding' --> [0.21, -0.49, 0.85, 0.14, ...] <-- similar! 'The dog ate the bone' --> [-0.67, 0.34, -0.12, 0.91, ...] <-- different Semantic search: find documents similar to a query by comparing vectors (cosine similarity or dot product)
```

Vector DB	Best For	Free Tier?
Pinecone	Production RAG apps, managed service	Yes (starter)
Chroma	Local development, open-source, Python-native	Yes (fully free)
Weaviate	Hybrid search (keyword + semantic)	Yes
FAISS (Meta)	Fast local similarity search, research	Yes (open source)



Vector DB	Best For	Free Tier?
pgvector	Vector search inside PostgreSQL database	Yes (open source)

6.7 Multimodal AI

Multimodal AI can process and generate multiple types of data — text, images, audio, and video — within a single model. This mirrors human intelligence more closely.

Capability	What it does	Model Examples
Text → Image	Generate images from text description	DALL-E 3, Midjourney, Stable Diffusion
Image → Text	Describe, caption, analyse images	GPT-4o Vision, Claude, Gemini Pro Vision
Text → Audio	Convert text to natural speech	ElevenLabs, OpenAI TTS, Google TTS
Audio → Text	Transcribe spoken language to text	OpenAI Whisper, Google Speech-to-Text
Text → Video	Generate video clips from text	Sora (OpenAI), Runway Gen-3, Kling
Text → Code	Write, debug, explain code	GitHub Copilot, GPT-4o, Claude, Gemini
Any → Any	True multimodal: text+image+audio in and out	GPT-4o, Gemini 1.5, Claude 3 Opus

Part 7 — AI Tools Landscape

7.1 Text & Chat AI

Tool	Company	Best For	Free Tier?
ChatGPT	OpenAI	General purpose; massive plugin ecosystem; GPT-4o	Yes (GPT-4o mini)
Claude	Anthropic	Long documents; writing; safety; coding analysis	Yes (Claude Sonnet)
Gemini	Google	Google Workspace integration; 1M context; multimodal	Yes (Gemini 1.5 Flash)
Microsoft Copilot	Microsoft	Integrated into Windows, Office 365, Bing	Yes
Perplexity AI	Perplexity	AI search with real-time web citations	Yes (limited)
Meta AI	Meta	Built into WhatsApp, Instagram, Facebook	Yes
Grok	xAI (Elon Musk)	Real-time Twitter/X data; less filtered	Free on X/Twitter

7.2 Image Generation AI

Tool	Company	Style / Strength	Access
DALL-E 3	OpenAI	Accurate to prompt; great text rendering; safe	ChatGPT Plus / API
Midjourney	Midjourney	Stunning artistic quality; best aesthetic results	Discord / Web (\$10/mo)
Stable Diffusion	Stability AI	Open source; run locally; full control	Free (local) / API
Adobe Firefly	Adobe	Commercially safe; great for design	Adobe CC / Free tier
Ideogram	Ideogram	Best for images with text overlaid	Free tier available
Leonardo.ai	Leonardo	Game assets; character consistency	Free tier available
Kling AI	Kuaishou	High-quality realistic images and video	Free tier available

7.3 Audio & Speech AI

Tool	Purpose	Notable Feature
ElevenLabs	Text-to-speech & voice cloning	Most realistic voices; clone any voice in 1 minute
OpenAI Whisper	Speech-to-text transcription	Open source; 99 languages; extremely accurate
Suno AI	Music generation from text prompt	Full songs with vocals, instruments, lyrics
Udio	Music generation	High-quality; fine-grained style control
Adobe Podcast	Audio enhancement & transcription	AI removes background noise from recordings
Descript	Audio/video editing via text	Edit audio by editing its transcript like a document

7.4 Video Generation AI



Tool	Company	Capability
Sora	OpenAI	Text → realistic 1-minute HD video; physics understanding
Runway Gen-3	Runway	Text+image → video; motion brush; inpainting
Kling AI	Kuaishou	High-quality video generation; realistic motion
Pika	Pika Labs	Easy-to-use; great for short social media clips
HeyGen	HeyGen	AI avatars; video translation with lip sync in 40+ languages
Synthesia	Synthesia	Professional AI presenter videos for training/marketing
Invideo AI	InVideo	Full video creation from a text prompt; auto-voiceover

7.5 Code AI Tools

Tool	Company	Integration	Best For
GitHub Copilot	GitHub/Microsoft	VS Code, JetBrains, Vim	Inline code completion; whole-function generation
Cursor	Cursor	Standalone AI IDE (VS Code fork)	Codebase-aware chat; multi-file edits
Claude	Anthropic	Web / API / VS Code	Code review, explanation, architecture design
Replit AI	Replit	Replit online IDE	Beginners; instant deployment; AI pair programmer
ChatGPT	OpenAI	Web / API	Debugging, code explanation, algorithm design
Tabnine	Tabnine	All major IDEs	Privacy-focused; runs locally; team code style learning

7.6 Productivity & Office AI

Tool	What it Does
Microsoft 365 Copilot	AI in Word, Excel, PowerPoint, Outlook, Teams — summarise emails, create slides, analyse data
Google Workspace AI (Gemini)	AI in Gmail, Docs, Sheets, Slides — draft emails, summarise docs, create formulas
Notion AI	Summarise notes, generate content, translate text inside Notion workspace
Otter.ai	Real-time meeting transcription with speaker identification and action-item extraction
Google NotebookLM	Upload your own documents; AI becomes an expert on YOUR content specifically
Tome	AI-powered presentation builder — create full slide decks from a text prompt
Gamma	Professional presentations, documents, and websites generated from text prompt

7.7 AI Search Engines

Tool	Key Feature	Best Use
Perplexity AI	Real-time web search + cited AI answers	Research, fact-checking, current events
ChatGPT (with Search)	Web browsing integrated into chat	Conversational search with AI follow-up
Google AI Overviews	AI summary at top of Google search results	Quick answers to common queries
Bing AI (Copilot)	Deep web integration; image search; code	Windows users; Office integration



Tool	Key Feature	Best Use
You.com	Multiple AI modes: search, chat, code, write	Developer-friendly; privacy options
Kagi	Ad-free AI-enhanced search	Privacy-focused users; no tracking

Part 8 — Using AI Tools: Practical Guide

8.1 Getting Started with ChatGPT / Claude / Gemini

Step	Action	Tip
1. Account	Sign up at chatgpt.com / claude.ai / gemini.google.com	Free tiers are sufficient for learning
2. First prompt	Type a clear, specific question or task	Start with something you know well so you can verify
3. Iterate	If response isn't right, refine your prompt	Say: 'That's good but make it shorter / more detailed'
4. Use roles	Assign an expert persona for better results	'Act as a Python teacher explaining to beginners'
5. Formatting	Request specific output formats	'Give me a table' / 'Use bullet points' / 'As JSON'
6. Verify	Cross-check important facts from AI responses	Never trust AI alone for medical, legal, or critical info

8.2 Image Generation — Midjourney & DALL-E Tips

Writing effective image prompts:

Basic structure: [Subject] + [Style] + [Setting] + [Lighting] + [Camera/Lens] + [Mood] Example (weak): 'A dog' Example (strong): 'A golden retriever puppy sitting in a sunlit garden, watercolour painting style, soft morning light, shallow depth of field, warm and joyful mood' Useful style keywords: photorealistic, 4K, cinematic, oil painting, anime, sketch, watercolour, studio lighting, golden hour, minimalist, vintage, cyberpunk, isometric

Midjourney commands: --ar 16:9 (aspect ratio) --style raw --v 6 (version) --no text (exclude text) /imagine [prompt]

8.3 GitHub Copilot — AI Coding Assistant

- Install in VS Code via the Extensions marketplace — search 'GitHub Copilot'.
- Sign in with your GitHub account (free for students via GitHub Education Pack).
- Type a comment describing what you want — Copilot suggests the code.
- Press Tab to accept a suggestion, Esc to reject, Alt+] for next suggestion.
- Use Copilot Chat (sidebar) to ask questions about your code or request explanations.
- Works best with clear, descriptive function names and comments before writing code.

Example — Copilot in action:

```
# Calculate the factorial of n using recursion
def factorial(n):
# Copilot auto-completes here:
    if n == 0 or n == 1: return 1
    return n * factorial(n - 1)
```

8.4 Google NotebookLM — AI on Your Documents

NotebookLM lets you upload your own documents (PDFs, Google Docs, websites, YouTube videos) and then chat with an AI that is an expert ONLY on your content. It cites every answer back to the specific source document and page. Perfect for studying, research, and analysis.

- Go to notebooklm.google.com — free with a Google account.
- Upload up to 50 sources: PDFs, Docs, slides, websites, YouTube URLs.

- Ask questions: 'Summarise this PDF' / 'What does Chapter 3 say about X?'
- Generate study guides, FAQs, briefing documents, and timelines from your sources.
- Audio Overview feature: generates a 10-minute podcast discussion about your documents.
- Share notebooks with collaborators for team research.

8.5 Building an AI Workflow

Combine multiple AI tools into a workflow to dramatically accelerate any task:

EXAMPLE WORKFLOW: Create a professional presentation Step 1: RESEARCH Perplexity AI --> 'Summarise the top 5 benefits of AI in education' Save the results Step 2: STRUCTURE ChatGPT/Claude --> 'Create a 10-slide presentation outline on AI in education' Based on the Perplexity research Step 3: CONTENT Claude --> 'Write speaker notes for each slide (50 words each)' Step 4: VISUALS DALL-E / Midjourney --> Generate custom images for each slide Step 5: PRESENTATION Gamma.app --> 'Create professional slides from this outline' OR paste into Google Slides / PowerPoint Total time with AI: ~20 minutes vs. Without AI: ~4 hours

Part 9 — Ethics, Safety & Responsible AI

9.1 Bias in AI

AI models learn from human-generated data — and human data contains human biases. These biases get baked into the model and can cause unfair, discriminatory, or harmful outputs.

Type of Bias	Example	Impact
Training Data Bias	Medical AI trained mostly on Western patients — less accurate for others	Healthcare disparities
Representation Bias	Image models associate certain jobs with specific genders	Reinforces stereotypes
Historical Bias	Hiring AI trained on historical decisions — replicates past discrimination	Unfair hiring
Confirmation Bias	Recommendation algorithms show you content that confirms your beliefs	Filter bubbles, polarization
Measurement Bias	Facial recognition less accurate on darker skin tones	Wrong arrests, misidentification

9.2 Deepfakes & Misinformation

AI can now generate hyper-realistic fake videos, audio recordings, and images of real people saying or doing things they never did. This technology poses serious risks to trust, privacy, and democratic processes.

- **How to spot deepfakes:** Look for unnatural eye blinking, mismatched lighting, blurry edges around face, audio-lip sync issues, and use detection tools.
- **Verification tools:** Google Fact Check, Snopes, Reuters Fact Check, InVID video verification browser extension.
- **AI watermarking:** C2PA (Content Credentials) standard — AI-generated images carry invisible metadata identifying them as AI-created. Adobe, OpenAI, Microsoft support this.

Never share or spread media without verification. 'Seeing' is no longer 'believing' in the age of generative AI.

9.3 Privacy Concerns

- When you type into ChatGPT or other AI tools, your conversations may be stored and used to train future models — unless you opt out in settings.
- Never share personal identification (Aadhaar, PAN, passwords, bank details) with AI tools.
- Be cautious sharing confidential business information, client data, or trade secrets.
- For sensitive work, use enterprise tiers with privacy guarantees, or run local models.
- EU AI Act and India's DPDP Act impose legal requirements on AI data handling.

9.4 Job Impact & the Future of Work

AI will change the nature of work significantly — automating some jobs while creating new ones. The key insight: AI replaces *tasks*, not necessarily entire jobs. Workers who learn to use AI as a tool will be more productive than those who don't.



Category	Jobs at Higher Risk	Jobs More Resilient
Most at risk	Data entry, basic content writing, repetitive coding, simple translation	
Evolving significantly	Accounting, legal research, customer service, medical imaging	Jobs still need human oversight and judgement
Growing due to AI	AI trainers, prompt engineers, AI ethicists, AI product managers	New roles emerging rapidly
Highly resilient	Skilled trades (plumbing, electrician), nursing, teaching, social work	Require physical dexterity, empathy, human connection

The most valuable skill in the AI era: knowing when to use AI, when not to, and how to critically evaluate its outputs. AI literacy is the new digital literacy.

9.5 AI Safety Principles & Responsible Use

Principle	Meaning	Example
Transparency	Be clear when AI is involved in creating content	Label AI-generated images and text
Fairness	Ensure AI treats all people equitably	Test models across different demographics
Accountability	Humans remain responsible for AI outputs	Don't blame the AI for harmful decisions
Privacy	Protect personal data used in AI systems	Follow GDPR/DPDP data minimisation rules
Reliability	AI systems should work consistently and safely	Test thoroughly before deploying
Human Oversight	Keep humans in the loop for important decisions	AI recommends; human decides



Part 10 — AI Careers & Next Steps

10.1 Career Paths in AI

Role	What You Do	Key Skills Needed
ML Engineer	Build, train, deploy ML models in production	Python, TensorFlow/PyTorch, MLOps, cloud
Data Scientist	Analyse data, build models, extract insights	Python, statistics, SQL, visualisation
AI/ML Researcher	Advance the science — publish new techniques	PhD, maths, deep learning theory
Prompt Engineer	Design optimal prompts for LLM-powered products	LLM knowledge, domain expertise, iteration
AI Product Manager	Define AI product strategy and roadmap	Product sense, technical literacy, user research
AI Ethics Specialist	Identify and mitigate AI harms and biases	Policy, law, social science, technical basics
Data Engineer	Build pipelines to collect and prepare AI training data	SQL, Spark, Airflow, cloud platforms
AI Solutions Architect	Design AI systems for enterprise clients	Cloud (AWS/GCP/Azure), APIs, system design
No-Code AI Builder	Build AI applications using tools without coding	Prompt engineering, Zapier, Make, n8n, Bubble

10.2 Skills to Learn — Roadmap

Foundation (for everyone):

- Understand AI concepts: ML, deep learning, LLMs, RAG, agents (these notes!).
- Prompt engineering — practice daily with ChatGPT, Claude, and Gemini.
- Python basics — the language of AI (variables, loops, functions, libraries).
- Data literacy — understand how to read and interpret data and statistics.

Intermediate (for technical roles):

- Python libraries: NumPy, Pandas, Matplotlib, Scikit-learn, Hugging Face Transformers.
- Deep learning frameworks: PyTorch (preferred) or TensorFlow/Keras.
- Cloud platforms: AWS SageMaker, Google Vertex AI, or Azure ML.
- APIs and integration: OpenAI API, REST APIs, LangChain, basic web development.

Advanced (for research/specialisation):

- Linear algebra, calculus, probability, and statistics — the mathematical foundations.
- Research papers: read ArXiv (arxiv.org) — 'Attention Is All You Need', GPT papers, RLHF.
- Fine-tuning, LoRA, PEFT — customising pre-trained models efficiently.
- MLOps: model deployment, monitoring, CI/CD for ML systems.

10.3 Best Learning Resources

Resource	Type	Level	Link / Where to Find
fast.ai — Practical Deep Learning	Video course	Beginner–Intermediate	fast.ai (free)



Resource	Type	Level	Link / Where to Find
DeepLearning.AI (Andrew Ng)	Video courses	All levels	Coursera / deeplearning.ai
Google Machine Learning Crash Course	Interactive course	Beginner	developers.google.com/ml
Hugging Face NLP Course	Hands-on course	Intermediate	huggingface.co/learn
3Blue1Brown — Neural Networks	Animated YouTube series	Beginner	YouTube: 3blue1brown
Andrej Karpathy — Neural Nets from scratch	YouTube series	Intermediate	YouTube: karpathy
CS229 Stanford — ML	Full university course	Intermediate–Advanced	YouTube / Stanford
Kaggle Learn + Competitions	Practical exercises	All levels	kaggle.com (free)
Papers With Code	Research papers + implementations	Advanced	paperswithcode.com
The Batch (Andrew Ng newsletter)	AI news weekly	All levels	deeplearning.ai/the-batch

10.4 Suggested Learning Projects

Project	Skills Practiced	Difficulty
Build a chatbot using the OpenAI API in Python	API calls, Python, prompt design	Beginner
Train an image classifier (cats vs dogs) using Keras	Deep learning, CNN, data loading	Beginner–Intermediate
Build a RAG app — chat with your own PDF documents	LangChain, embeddings, vector DB	Intermediate
Fine-tune a small LLM on a custom dataset	Hugging Face, LoRA, PEFT	Intermediate–Advanced
Create an AI agent that browses the web and answers questions	LangChain agents, tools, planning	Intermediate
Build an automated news summariser using LLMs + cron	APIs, scheduling, prompt chaining	Intermediate
Participate in a Kaggle competition (start with Titanic)	Data science, ML pipeline, submission	Beginner
Create a Streamlit web app for any AI model	Python, Streamlit, API integration	Beginner–Intermediate



Appendix — Prompt Engineering Quick Reference

Prompt Templates for Common Tasks

Task	Template
Explain a concept	"Explain [concept] to a [beginner / 10-year-old / expert]. Use a real-life analogy. Keep it under [N] words."
Summarise text	"Summarise the following text into [5 bullet points / 3 paragraphs / 1 sentence]: [paste text]"
Compare options	"Compare [A] and [B] in a table with columns: Feature, [A], [B], Winner. Focus on: [criteria]"
Write email	"Write a [formal/friendly] email to [recipient] about [topic]. Tone: [tone]. Length: under [N] words."
Debug code	"You are a senior [language] developer. Find bugs and suggest fixes for this code: [paste code]"
Generate ideas	"Give me [N] creative ideas for [topic]. Format: numbered list. Each idea in one sentence."
Create quiz	"Create a [N]-question multiple choice quiz on [topic] for [audience]. Include answers at the end."
Rewrite text	"Rewrite the following text to be [clearer / shorter / more professional / simpler]: [paste text]"
Step-by-step	"Explain how to [task] step by step. Assume I have [no / basic / some] knowledge of [topic]."
Pros and cons	"List the pros and cons of [topic / decision] in two columns. Be balanced and objective."
Role play	"You are a [role/expert]. I will ask you questions about [domain]. Stay in character throughout."
Data analysis	"Analyse this data and identify: key trends, anomalies, and 3 actionable recommendations: [data]"

Key AI Terms — One-Line Definitions

Term	One-Line Definition
AI	Computer systems that perform tasks requiring human-like intelligence.
Machine Learning	AI that learns patterns from data without explicit programming.
Deep Learning	ML using multi-layer neural networks; excels at images, text, speech.
LLM	Massive text-trained model that generates human-quality language.
Token	Basic unit of text (~0.75 words) processed by an LLM.
Prompt	The input text you give to an AI model.
Prompt Engineering	Crafting effective prompts to get optimal AI outputs.
Temperature	Randomness control: low = focused, high = creative.
Context Window	Maximum text length the model can process at once.
Hallucination	AI confidently producing plausible but factually wrong information.
RAG	Technique giving AI access to external documents at query time.
Embedding	Numerical vector representation of text capturing semantic meaning.
Fine-Tuning	Additional training to specialise a pre-trained model.
Agent	AI that can take actions (web search, code execution) to achieve goals.
Transformer	Neural network architecture (2017) underpinning all modern LLMs.



Term	One-Line Definition
RLHF	Training AI using human feedback ratings to align with human values.
Multimodal	AI that handles multiple data types: text, image, audio, video.
Zero-Shot	Model performs a task without being given examples first.
Few-Shot	Model is given 2–5 examples before performing the actual task.
Chain-of-Thought	Asking AI to reason step by step before giving a final answer.